UNIVERSIDAD TECNOLÓGICA DEL PERÚ (UTP)
Facultad de Ingeniería • Carrera de Ingeniería de Sistemas e Informática
Curso: Integrador I: Sistemas Software (Sección 20065) • Entregable Académico Oficial S11

INFORME TÉCNICO: APLICACIÓN DE GIT Y GITHUB EN EL PROYECTO INTEGRADOR

Evidencias de Control de Versiones, Ramificación GitFlow y Trazabilidad de Entregables (S11)

Estudiante: Ronny Fernando Mendoza Paucar	**Código UTP:** U25302314
Proyecto Integrador: SIIP - Sistema Inteligente de Inventario y Predicción	**Caso de Negocio:** Cadena Farmacias Peruanas (InkaFarma / Mifarma)
Repositorio GitHub Oficial: https://github.com/Fern-411/Farmacias	**Modalidad de Entrega:** Individual (Formato PDF requerido)

1. Introducción y Justificación Estratégica en el Contexto Farmacéutico

El presente informe formaliza la adopción de **Git** (como sistema distribuido de control de versiones) y **GitHub** (como plataforma de alojamiento y colaboración) en el proyecto **Sistema Inteligente de Inventario y Predicción (SIIP)** para **Farmacias Peruanas (InkaFarma y Mifarma)**. En proyectos informáticos de alta criticidad en salud pública, el versionamiento no es únicamente una herramienta de respaldo; representa un **imperativo técnico y regulatorio** alineado con las directrices de la **DIGEMID (Manual de Buenas Prácticas de Almacenamiento - R.M. 833-2015)** y la **SUNAT (Guías de Remisión Electrónica GRE)**, las cuales demandan una trazabilidad rigurosa, auditable y sin alteraciones de los módulos que administran el stock y los despachos inter-sucursales.

A través de Git y GitHub se han garantizado cuatro beneficios fundamentales para el Proyecto Integrador: (1) **Inmutabilidad criptográfica** mediante hashes SHA-1 en cada confirmación (commit); (2) **Reproducibilidad técnica** de compilaciones multi-módulo (Maven Spring Boot y React Vite); (3) **Segregación segura de secretos** y llaves privadas bajo la Ley N° 29733; y (4) **Gestión organizada de entregables** mediante el flujo de trabajo GitFlow.

2. Arquitectura de Ramificación y Metodología GitFlow

Para estructurar la construcción progresiva del software sin desestabilizar las versiones entregables, se adoptó el estándar **GitFlow**. Bajo esta metodología, la rama main se reserva exclusivamente para versiones estables y aprobadas, mientras que el desarrollo de cada caso de uso funcional se aísla en ramas feature/* y las pruebas automatizadas en test/*, consolidándose únicamente mediante Pull Requests (PRs) con revisión de código previa.

Esquema de Flujo GitFlow Implementado en el Proyecto SIIP:

```
[main] <==================== Pull Request Merge (Revisado & Probado) ====================<
  ▲
     ▲
  ■■■ feature/01-arquitectura-backend-docker ➔ Spring Boot multi-módulo, Docker Compose, red privada
  ■■■ feature/02-auth-service-rbac ➔ ms-auth, PostgreSQL 16, BCrypt, 5 roles farmacéuticos y Guava
  ■■■ feature/03-inventario-geocercano ➔ Caso CU01: Algoritmo Haversine, minimapa y red de 15 sedes
  ■■■ feature/04-gestion-productos-kardex ➔ Catálogo comercial vs genéricos DCI, Cloudflare R2 y kardex
  ■■■ feature/05-gestion-sedes ➔ Cobertura nacional (Lima, Cusco, Arequipa, Trujillo, Chiclayo, Huancayo)
  ■■■ feature/06-alertas-predictivas ➔ Predicción de quiebre a 7 días y control de lotes caducidad FEFO
  ■■■ feature/07-traslados-gre-sunat ➔ Caso CU03: Guías de Salida GRE con QR/HASH y Actas DIGEMID BPA
  ■■■ feature/08-ui-ux-fluid-layout ➔ Layout widescreen responsivo, alto contraste y modales ergonómicos
  ■■■ test/09-qa-pruebas-sistema ➔ Suite automatizada E2E, capturas funcionales y validaciones TDD
```

Tabla Detallada de Ramas Creadas y Versionadas en GitHub

A continuación se detalla la matriz de ramas del repositorio https://github.com/Fern-411/Farmacias, evidenciando el trabajo incremental y ordenado por módulos:

Nombre de Rama	Alcance Técnico / Entregable Específico	Estado en GitHub
main	Rama principal protegida. Contiene la versión consolidada con microservicios Spring Boot, frontend Vite React y base de datos PostgreSQL 16 con 8 tablas.	**Activa / Sincronizada**
feature/01-arquitectura-backend-docker	Estructura multi-módulo Maven, shared-kernel, API Gateway Spring Cloud y docker-compose.yml.	Mergeado (PR #1)
feature/02-auth-service-rbac	Microservicio ms-auth, autenticación PostgreSQL, hash BCrypt, 5 roles y bloqueo a 5 intentos con Guava.	Mergeado (PR #2)
feature/03-inventario-geocercano	Consulta geocercana con fórmula Haversine (<10 km), geolocalización de 15 sedes y visor de minimapa.	Mergeado (PR #3)
feature/04-gestion-productos-kardex	Catálogo farmacéutico con marcas vs genéricos DCI, bucket Cloudflare R2 y asientos de kardex.	Mergeado (PR #4)
feature/05-gestion-sedes	Gestión de sucursales en Lima Metropolitana y sedes macrorregionales con filtro por departamento.	Mergeado (PR #5)
feature/06-alertas-predictivas	Algoritmo de predicción de rotura a 7 días, control de lotes con caducidad FEFO semaforizada (Rojo/Verde).	Mergeado (PR #6)
feature/07-traslados-gre-sunat	Flujo de traslados express: emisión de GRE Remitente SUNAT con código QR/HASH y Actas BPA DIGEMID.	Mergeado (PR #7)
feature/08-ui-ux-fluid-layout	Diseño fluido widescreen, barra sticky, cierre de modales por clic exterior y tecla Escape.	Mergeado (PR #8)
test/09-qa-pruebas-sistema	Suite automatizada de pruebas funcionales, capturas de pantalla de evidencias y tests unitarios TDD.	Mergeado (PR #9)

3. Buenas Prácticas de Versionamiento y Seguridad de Datos (Ley N° 29733)

En cumplimiento de los principios de confidencialidad y el estándar de seguridad de la información, se configuró un archivo .gitignore de nivel empresarial para impedir que llaves criptográficas, credenciales de conexión o archivos binarios se publiquen en GitHub:

Configuración de Exclusiones en .gitignore (Extracto Real del Repositorio):

```
# PROTECCIÓN DE SECRETOS, CREDENCIALES Y VARIABLES DE ENTORNO (Ley 29733)
.env # Variables locales de microservicios y bases de datos
.env.local # Sobrescrituras locales de desarrollador
*.pem / *.key # Certificados y llaves privadas SSL/TLS
credentials.json # Llaves de servicios externos (Cloudflare R2)

# EXCLUSIÓN DE BINARIOS Y ARTEFACTOS GENERADOS EN LA COMPILACIÓN
node_modules/ # Dependencias de NodeJS (Vite React)
backend/target/ # Binarios compilados de Spring Boot (*.jar, *.class)
logs/ / *.log # Archivos de auditoría operativa de Logback
docker-data/ # Volúmenes de datos locales de PostgreSQL
```

Convención de Mensajes de Confirmación (Conventional Commits v1.0.0)

Se aplicó la especificación semántica **Conventional Commits** para garantizar que cada registro en el historial describa con exactitud la naturaleza del cambio técnico:

Prefijo	Significado	Ejemplo Real en el Repositorio GitHub
feat(...)	Nueva característica funcional	feat(auth): migracion a postgresql, 5 roles indispensables y control rbac con guava
fix(...)	Corrección de bug o defecto	fix(modales): estandarizar cierre por clic exterior y tecla escape en todos los modales
docs(...)	Documentación y entregables	docs(pptx): generar diapositivas ejecutivas con evidencias del sistema para sustentacion UTP
test(...)	Pruebas unitarias o E2E	test(e2e): suite de capturas automatizadas de casos de uso CU01 a CU05
chore(...)	Configuraciones de build/entorno	chore(build): sincronización final de artefactos y configuración de entorno

4. Evidencias de Comandos Git y Flujo de Trabajo en Consola

A continuación se documenta la ejecución real de comandos en la terminal de desarrollo PowerShell, acreditando la interacción con Git y la sincronización con el repositorio remoto de GitHub:

Consola PowerShell — Transcripción Fidedigna de Comandos y Respuestas Git:

```
# 1. ESTADO DE SINCRONIZACIÓN DE LA RAMA PRINCIPAL
PS D:\UTP\Farmacias> git status
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean

# 2. HISTORIAL COMPACTO DE COMMITS CON HASHES SHA-1 Y AUTORES
PS D:\UTP\Farmacias> git log --oneline --decorate -n 6
c654649 (HEAD -> main, origin/main) feat(inventory): persistencia real de inventario, traslados express y resolucion
de quiebre amoxidal 500mg
3f0a211 feat(auth): migracion a postgresql, 5 roles indispensables y control rbac con guava
377f276 feat(avance-3): integrar PostgreSQL con 8 tablas, Guava (bloqueo a 5 intentos), Apache POI, Apache Commons,
Logback y presentacion ejecutiva
dfc3a09 Merge pull request #9 from Fern-411/test/09-qa-pruebas-sistema
51fe99d Merge pull request #8 from Fern-411/feature/08-ui-ux-fluid-layout
6deba5b Merge pull request #7 from Fern-411/feature/07-traslados-gre-sunat

# 3. INSPECCIÓN DE RAMAS REMOTAS EN GITHUB
PS D:\UTP\Farmacias> git branch -r
origin/HEAD -> origin/main
origin/feature/01-arquitectura-backend-docker
origin/feature/02-auth-service-rbac
origin/feature/03-inventario-geocercano
origin/feature/04-gestion-productos-kardex
origin/feature/05-gestion-sedes
origin/feature/06-alertas-predictivas
origin/feature/07-traslados-gre-sunat
origin/feature/08-ui-ux-fluid-layout
origin/test/09-qa-pruebas-sistema
origin/main

# 4. VERIFICACIÓN DEL REPOSITORIO REMOTO ORIGIN
PS D:\UTP\Farmacias> git remote -v
origin https://github.com/Fern-411/Farmacias.git (fetch)
origin https://github.com/Fern-411/Farmacias.git (push)
```

5. Trazabilidad de los Entregables del Curso Integrador en GitHub

Cada hito evaluado por la cátedra a lo largo del semestre 2026 cuenta con su trazabilidad de código y artefactos en el repositorio:

Hito Académico / Entregable	Componentes Técnicos Versionados	Ubicación en el Repositorio
Entregable 1: Arquitectura y Modelado	Arquitectura en 4 Capas (MVC, DAO, SOLID, TDD). Modelos JPA, interfaces de servicio e inyección de dependencias Spring Boot.	/backend/service-auth/ /backend/shared-kernel/
Entregable 2: Microservicios y Contenedores	Orquestación Docker Compose (PostgreSQL, Auth, Inventory, Predictor, Gateway). Red privada farmacias-net y healthchecks.	/docker-compose.yml /backend/*/Dockerfile
Entregable 3: PostgreSQL, Librerías y Persistencia	Esquema relacional de 8 tablas con índices, hash BCrypt, **Google Guava** (bloqueo a 5 intentos), **Apache POI** (reportes Excel), **Apache Commons** y **Logback**.	/scripts/database-seed.sql /backend/.../logback-spring.xml
Evidencia de Pruebas y Sustentación	Presentación ejecutiva de 23 diapositivas widescreen 16:9 con capturas de los 5 casos de uso, matriz RBAC y justificación técnica.	PRUEBAS_SISTEMA_SIIP_FARMACIAS_AVANCE3.pptx

6. Conclusiones y Métricas Finales de Versionamiento

1. **Gobernanza y Trazabilidad Farmacéutica:** La adopción de Git y GitHub permitió registrar con exactitud cada ajuste al motor de inventarios y autenticación. El historial permite rastrear con precisión al autor, fecha y justificación técnica de cada cambio, satisfaciendo los criterios de auditoría de DIGEMID y SUNAT.

2. **Resolución Sistemática de Conflictos:** La aplicación de git pull --rebase y el trabajo en ramas temáticas independientes evitó colisiones de código entre microservicios, asegurando que la integración de bibliotecas complejas (Google Guava para bloqueo de fuerza bruta y Apache POI para reportes) se efectuara sin regresiones.

3. **Persistencia e Integridad del Software:** El versionamiento modular facilitó resolver de raíz la persistencia de stock en pantalla (resolviendo el quiebre de Amoxidal 500 mg entre recargas F5) y respaldar la autenticación real en PostgreSQL mediante scripts SQL versionados.

Métrica de Versionamiento	Valor Registrado en el Proyecto	Evidencia Técnica en GitHub
Ramas de Trabajo (Branches)	10 ramas (1 main + 9 feature/test)	Desglose modular de cada sprint del proyecto
Pull Requests Integrados	9 Pull Requests mergeados a main	Control de cambios y aprobación por pares
Estándar de Commits	Conventional Commits v1.0.0	Prefijos estructurados feat, fix, docs, test
Protección de Secretos	.gitignore con 32 exclusiones	Cero fugas de credenciales o claves privadas
Acceso Público al Repositorio	**100% Público y Accesible**	**https://github.com/Fern-411/Farmacias**

DECLARACIÓN DE AUTORÍA Y ENTREGA INDIVIDUAL:
El suscrito estudiante declara bajo juramento que el código fuente, la configuración del repositorio GitHub oficial, las ramas de trabajo, los commits registrados y la documentación técnica presentada en este informe corresponden a su participación activa y autoría en el desarrollo del Proyecto Integrador SIIP (Farmacias Peruanas), habiendo incorporado el uso sistemático de Git para asegurar la calidad y trazabilidad integral del software evaluado en la Semana 11.

Firma del Estudiante: _____
Ronny Fernando Mendoza Paucar | *Código UTP:* ***U25302314***
Carrera de Ingeniería de Sistemas e Informática — Universidad Tecnológica del Perú
Lima, Octubre de 2026

ENLACE DIRECTO AL REPOSITORIO GITHUB OFICIAL DEL PROYECTO:
https://github.com/Fern-411/Farmacias
Incluye el código completo, ramas de desarrollo, pull requests fusionados, Dockerfiles, scripts SQL y presentación PPTX.